INTERFAZ GRÁFICA

Segunda instancia evaluativa

Proyecto: Simulador de Ecosistema versión Swing — examen práctico grupal

Información general

- Examen práctico grupal (3 a 4 integrantes) pueden ser el mismo grupo de la **IE1**. ***No se aceptan trabajos individuales.***
- Entrega: 04 de noviembre de 2026
- Cada grupo parte de **su propio proyecto de la IE1**: no se entrega un proyecto nuevo, se transforma el que ya tienen.

Entregables obligatorios

> **CRÍTICO:** *la falta de cualquiera de los 3 entregables obligatorios anula la instancia evaluativa del alumno.*

1. Código del proyecto (obligatorio)

- El proyecto se entrega únicamente a través de un repositorio de GitHub. Se coloca el link al repositorio.
- El repositorio puede ser público o privado. Si es privado, deben darle acceso a la docente antes de la entrega.
- El trabajo se desarrolla usando Git desde el comienzo: el historial de commits debe reflejar el proceso real. No se acepta un proyecto subido de una sola vez ni como archivo comprimido.
- Todos los integrantes deben tener commits realizados, hechos desde su propia cuenta de GitHub (no se comparte una cuenta).
    - **Cuidado:** el integrante sin commits en el código pierde el 50% de la nota grupal.
- Los commits se evalúan como parte de la nota individual: que sean frecuentes, distribuidos desde que se publica esta consigna hasta el día de la entrega (no concentrados en el último día), con mensajes descriptivos y con una contribución proporcional al trabajo del grupo.
- Se tienen en cuenta solo los commits realizados hasta el inicio de la clase de entrega.
- El proyecto debe abrir y ejecutarse en NetBeans sin tocar nada: sin rutas absolutas y con la clase principal (main) en el paquete principal o, si no lo crean, en el controlador.

- Dentro del repositorio debe estar el link al archivo de Figma (en el README o en un archivo de texto aparte).

2. Video explicativo (obligatorio)

- Duración: entre 10 y 15 minutos.
- Todos los integrantes **con cámara encendida**.
- La explicación de cada integrante puede ser individual y luego unirse en un solo video.
- Cada integrante debe **mostrar su pantalla con el código** al momento de explicar.
- Se debe explicar: la funcionalidad implementada, el código técnico (indicando qué hace cada capa, modelo, vista, controlador y dao, en la parte que le tocó), las decisiones de diseño tomadas en Figma, los conflictos encontrados, cómo los resolvieron, y uso transparente de IA u otra documentación.
- Se evaluará especialmente la comprensión individual de cada parte del código implementada.
- Pueden usar Vimeo para capturar pantalla y rostro. Sin webcam: grabarse con el celular y unir con Canva.

3. Documentación (obligatorio)

- En la carpeta del proyecto debe haber una carpeta con capturas de los prompts utilizados, o un Word con los links a las conversaciones completas con la IA, o links de páginas web consultadas.
- Esta entrega es **INDIVIDUAL**. Deberán dejar claro de quién es cada captura.
- Alternativa: agregar los links de las conversaciones completas en un README.md.

README.md (opcional, recomendado)

- Instrucciones de ejecución (versión de NetBeans y de JDK, librerías, etc.)
- Estructura del proyecto: qué hay en cada paquete
- Integrantes y rol de cada uno
- Link a Figma
- Link al video explicativo
- Desafíos encontrados

- Uso de IA / herramientas externas

Objetivo del proyecto

Transformar el Simulador de Ecosistema de la primera instancia, que corría en la terminal, en una **aplicación de escritorio con interfaz gráfica en Swing**. La aplicación se organiza con arquitectura **MVC en paquetes separados**, guarda y retoma la simulación mediante un **DAO con archivos de texto plano (.txt)**, y su interfaz se diseña antes en **Figma** aplicando UX/UI.

La mecánica del juego **no cambia**: el jugador configura el ecosistema, observa cómo evoluciona turno a turno e interviene cada 3 turnos. Lo que cambia es cómo está organizado el código y cómo lo usa la persona.

> *Este proyecto aplica de forma integrada: diseño UX/UI, componentes y layouts de Swing, arquitectura MVC, DAO con archivos, y todo lo de POO de la IE1 (herencia, clases abstractas, interfaces, polimorfismo, encapsulamiento y colecciones).*

Mecánica del juego

Se mantiene **exactamente la de la IE1**. Se resume acá para tenerla a mano.

Configuración inicial

Ahora se ingresa en un formulario de la ventana (no con Scanner):

- Cantidad inicial de plantas (mínimo 5, máximo 30)
- Cantidad inicial de conejos (mínimo 2, máximo 15)
- Cantidad inicial de lobos (mínimo 1, máximo 5)
- Clima inicial
- Cantidad de turnos totales de la simulación (entre 10 y 50)

Cada turno, en orden

- Las plantas con suficiente energía se reproducen.
- Los conejos buscan plantas y comen (si no encuentran, pierden energía).
- Los lobos intentan cazar un conejo (probabilidad según la energía del lobo).
- Todas las entidades envejecen y gastan energía base.

- Las entidades sin energía mueren.
- Se muestra el estado del ecosistema: contadores y eventos del turno.
- Cada 3 turnos: el jugador puede intervenir o simplemente avanzar.

Clima y sus efectos

Se mantienen los climas y los efectos de la IE1 (Debe haber por lo menos 3 climas; quien tenga menos, completa lo que falte).

Intervención del jugador (cada 3 turnos)

- Cambiar el clima
- Agregar entidades (máximo 5 lobos en total en toda la simulación)
- Solo avanzar turno sin intervenir

Condiciones de fin

- Se alcanzó el número de turnos configurado.
- Se extinguieron los conejos, los lobos o las plantas (colapso del ecosistema).

Al finalizar se muestra un reporte completo de la simulación.

Clases obligatorias

> ***Libertad de implementación:*** *esta consigna define* ***qué debe cumplir cada parte****, no cómo programarla. Los nombres de las clases y de los métodos, la forma de organizar el código por dentro y el formato del archivo de texto son decisión del grupo. Lo que sí se evalúa es que cada capa cumpla con su responsabilidad y que el grupo pueda explicar sus decisiones en el video.*

El proyecto se organiza en cuatro paquetes (modelo, vista, controlador y dao), más un paquete principal opcional, con estas responsabilidades:

Paquete modelo

- Contiene las clases de la IE1 (entidades, jerarquía, interfaces, clima y ecosistema), con la herencia, el polimorfismo y el encapsulamiento que ya tenían.
- Aplica las reglas de la simulación y calcula los datos del reporte final.

- **No interactúa con el usuario**: sin Scanner, sin System.out.println y sin nada de Swing. Los eventos del turno quedan guardados para que otra capa los pueda pedir y mostrar.
- Tiene lo necesario para que el estado pueda guardarse y volver a armarse al cargar.

Paquete dao

- Tiene los métodos necesarios para **guardar** los datos de la simulación en un archivo .txt.
- Tiene los métodos necesarios para **leer** ese archivo y devolver los datos, de modo que la aplicación pueda mostrarlos y retomar la simulación.
- Usa archivos de texto plano y maneja los errores de entrada/salida.
- No usa Swing, no conoce la vista ni el controlador y no muestra mensajes: informa el resultado y el controlador decide qué hacer.

Paquete controlador

- Recibe las acciones de la vista y coordina al modelo y al DAO.
- **Valida los datos** que ingresa la persona (formato, rangos, campos vacíos) y le avisa a la vista cuando algo está mal.
- Se encarga de que lo que se muestra en pantalla esté siempre actualizado: le entrega a la vista los datos actuales.
- No tiene código Swing (la única excepción es la del main descripta en el paquete principal).

Paquete vista

- Muestra los datos y recoge las acciones de la persona. **No tiene lógica de negocio**: no decide, no calcula ni aplica reglas de la simulación.
- Cuando la persona hace algo (un clic, un dato ingresado), la vista se lo pasa al controlador.
- Recibe los datos a mostrar desde el controlador; no los busca por su cuenta en el modelo ni en archivos.
- La interfaz se construye con el diseñador visual de NetBeans. No se aceptan componentes creados ni configurados por código (por ejemplo: new JButton(), new JTable(), new JLabel(), add(), setLayout(), setBounds()).

- Por código solo se trabaja con los componentes que ya existen en el diseñador: cargar datos, agregar filas a una tabla, habilitar o deshabilitar componentes, cambiar de pantalla y mostrar mensajes.
- Si hay algo puntual que el diseñador no permite resolver, se puede hacer por código de forma acotada, explicando en el video qué es y por qué.

Paquete principal (opcional)

- Contiene el `main`: crea las piezas, las conecta y muestra la ventana.
- Solo si desean separar la lógica de los inicializadores. Si no crean este paquete, el main se ubica en el paquete controlador.
- Excepción para el main en el controlador: se permite usar Swing únicamente para crear y mostrar la ventana (JFrame) que contiene la aplicación (por ejemplo: new JFrame, add, pack, setLocationRelativeTo, setDefaultCloseOperation y setVisible). Solo esas líneas.
- En el controlador siguen sin aceptarse JOptionPane, listeners o eventos (ActionListener, MouseListener, etc.) ni el manejo de componentes (tablas, listas, botones, textos). Todo eso es trabajo de la vista.

Funcionalidades a implementar

1. Diseño en Figma (antes de programar)

- El diseño presenta **todas las vistas de la aplicación** (por ejemplo, configuración, simulación en curso, reporte final y cualquier otra que el grupo agregue).
- Se incluyen los estados importantes de la interfaz: cuando no hay datos, cuando hay un error y cuando se confirma una acción.
- **La estética es libre**; lo que importa es que las decisiones estén justificadas con los principios vistos en clase (al menos 3 decisiones explicadas) y que el diseño sea coherente entre pantallas.
- Breve descripción de quién usa la aplicación y para qué (persona y escenario).
- La aplicación se parece al diseño. Si algo cambió al programarla, se explica por qué.
- El link a Figma va dentro del proyecto y una captura de las pantallas va en la documentación.

2. Configuración inicial

- La persona configura la simulación desde la ventana, con las mismas reglas y rangos de la IE1.
- Los datos ingresados se validan en el controlador y los errores se muestran en la vista de forma clara.

3. Desarrollo de la simulación

- La persona avanza la simulación turno a turno desde la interfaz.
- En todo momento se ve el estado del ecosistema (turno, clima, cantidad de cada población) y las entidades vivas.
- Se ven los eventos de cada turno de forma legible (quién comió a quién, quién murió, quién nació).
- La interfaz da respuesta visible a cada acción y contempla el caso en que todavía no hay una simulación en curso.

4. Intervención del jugador

- Cada 3 turnos la persona puede cambiar el clima, agregar una entidad o simplemente avanzar.
- Se mantienen las reglas de la IE1 (por ejemplo, el máximo de 5 lobos en total). Las reglas las aplica el modelo y el controlador; la vista solo las muestra.
- La interfaz deja claro cuándo se puede intervenir y cuándo no.

5. Guardado y retoma de la simulación

- La simulación se guarda en un archivo de texto plano y se puede **retomar**: al volver a abrir la aplicación, se muestran los datos guardados y la simulación continúa donde estaba.
- El grupo decide qué guarda, en qué formato y cuándo lo guarda, pero debe ser suficiente para que la simulación y el reporte final continúen sin perder información.
- El archivo se ubica dentro de la carpeta del proyecto, sin rutas absolutas.
- La aplicación no se rompe si el archivo no existe o si tiene datos inválidos.
- Si algo falla al guardar o cargar, la persona se entera por un mensaje de la interfaz.
- Debe existir una forma clara de empezar una simulación nueva, con confirmación si hay una en curso.

6. Reporte final

Un reporte **completo y prolijo**, accesible al finalizar la simulación (por turnos completos o por colapso):

- Causa de fin (turnos completos o colapso, especificando cuál población se extinguió).
- Turno de mayor actividad.
- Entidad más longeva de cada tipo.
- Lobo con más cacerías exitosas.
- Total de nacimientos y muertes por tipo a lo largo de toda la simulación.
- **Prolijo** significa: los datos aparecen ordenados, rotulados y legibles dentro de la ventana, sin texto cortado ni información mezclada.

7. Requisitos obligatorios para aprobar

- **Arquitectura:** los paquetes modelo, vista, controlador y dao (principal es opcional) separados, cada uno con la responsabilidad descripta. No se aceptan proyectos con lógica, interfaz y acceso a archivos mezclados.
- **POO de la IE1** se mantiene: herencia, clases abstractas, interfaces, polimorfismo y encapsulamiento.
- **Interfaz Swing funcional** construida con el diseñador visual de NetBeans, sin crear ni configurar componentes por código, que use componentes adecuados al tipo de dato que muestra (datos en tabla, listas de eventos, formularios, botones con acciones reales, scroll donde haga falta).
- **Más de una pantalla** con navegación entre ellas.
- **Layouts usados con criterio**: no se aceptan componentes ubicados sin layout. El diseño libre que ofrece el diseñador por defecto es válido; no se acepta Absolute Layout.

CRÍTICO: decisiones de arquitectura que anulan el criterio correspondiente

- Acceso a archivos desde la vista o desde un evento de botón.
- Lógica de la simulación (reglas, cálculos, decisiones) dentro de la vista.
- El modelo o el DAO con código de Swing.
- El controlador con código de Swing (incluidos `JOptionPane`, listeners y manejo de componentes), salvo las líneas del main que crean y muestran el JFrame cuando el main está en el controlador.
- `Scanner` o `System.out.println` dentro del modelo.
- La vista accediendo directamente al modelo para buscar sus datos.
- Componentes de la interfaz (botones, tablas, paneles, etc.) creados o configurados por código en lugar de con el diseñador.

Bonus (puntos extras)

Los puntos obtenidos en los ítems extra se suman al puntaje final del proyecto para mejorar la nota global de la IE.

Prototipo navegable en Figma

- Conectar las pantallas con interacciones reales (configuración → simulación → reporte).

Menú superior (JMenuBar)

- Menú con al menos: Simulación (nueva) y Reportes (ver reporte). Debe llamar al controlador, igual que los botones.

Íconos o imágenes

- Usar `ImageIcon` para diferenciar visualmente plantas, conejos y lobos (en la tabla, en los contadores o en los eventos).

Estadísticas de población

- Historial de población por turno y turno de máximo y mínimo de cada tipo, mostrado en la interfaz.
- Si la guardan, el historial también tiene que persistir en el archivo.

Planta Venenosa en la interfaz

- Si la implementaron en IE1: mostrarla diferenciada en la tabla y registrar en los eventos cuando un conejo la come.

Ejemplo de flujo y de pantalla

Una forma posible de comunicar las capas al presionar «Avanzar turno» (orientativa, cada grupo puede resolverlo distinto):

```
① Vista       → el botón llama a controlador.avanzarTurno()
② Controlador → pide al modelo que procese el turno
③ Modelo      → procesarTurno() ← devuelve la lista de eventos del turno
④ Controlador → pide al DAO que guarde el estado
⑤ DAO         ← devuelve true/false según si pudo guardar
⑥ Controlador → ordena a la Vista actualizar tabla, contadores y eventos
                (si guardar falló: vista.mostrarError(...))
⑦ Controlador → si hubo colapso o se cumplieron los turnos,
                ordena a la Vista mostrar el final
```

Distribución orientativa de la pantalla de simulación (cada grupo la adapta a su diseño de Figma):

Zona	Contenido sugerido
Norte	Contadores (turno, clima, plantas, conejos, lobos).
Centro	Tabla de entidades vivas.
Este	Lista de eventos del turno.
Sur	Botón Avanzar turno, panel de intervención y mensajes de estado.

Distribución de puntaje de la 2° IE

- Parte grupal: 70% del puntaje global
- Parte individual: 30% del puntaje global

Escala de valoración

Nota	Puntaje
1	01 – 24
2	25 – 39
3	40 – 54
4 (regulariza)	**55 – 61**
5	**62 – 66**

Nota	Puntaje
6	67 – 72
7 (accede a IEFI)	73 – 79
8	80 – 87
9	88 – 96
10	97 – 100

Modalidad de entrega

El grupo se presentará en el aula, en el horario de la clase, el día de la instancia evaluativa. Los primeros 10 a 20 minutos de la clase se destinarán a la entrega del proyecto.

Cada integrante deberá completar un formulario individual con sus datos y link al repositorio donde estén alojados el proyecto, el video y la documentación. La entrega queda asentada únicamente a través de ese formulario.

Inmediatamente después de finalizado ese plazo, la docente descargará el repositorio entregado y el contenido de las carpetas indicadas.

> *Cualquier actualización, corrección o carga posterior (incluyendo prompts, videos reemplazados por errores, u otro archivo) no será tenida en cuenta bajo ningún concepto: se corregirá únicamente lo que se haya descargado dentro de ese plazo.*

Fuera de ese tiempo, no se aceptará ningún proyecto, ningún video, ni ninguna documentación. La docente no avisará por problemas de acceso a los links: es responsabilidad del grupo que todo esté disponible y con los permisos correspondientes antes del inicio de la clase.

Ausencias: cada integrante debe estar presente y completar el formulario dentro del plazo establecido. La docente tomará asistencia y no podrá cargar las notas de aquellos alumnos que se encuentren ausentes en el día del examen. La única excepción es presentar, dentro de la semana posterior a la entrega, un certificado médico u otra justificación válida ante Atención Académica. En ese caso, la corrección de su participación se coordinará por separado con la docente.

Pasados los primeros 20 minutos, la clase continuará con el programa normal con los contenidos que entrarán en la IEFI.